AI API 명세서

면접 역질문 서비스(askback24)가 AI 서버에 **질문 문장 쓰기**와 **1차 면접 답 판정**(1·2번), **공고 분석**과 **업무 카드 초안**(3·4번)을 요청하는 API입니다. 우리 서버(Vercel 함수)가 부르고, AI 파이프라인 담당이 AI 서버에 만듭니다. 질문 개수·순서·유형·근거는 우리 규칙이 정해 요청에 담고, AI 서버는 문장과 판정만 돌려줍니다.

> AI 서버는 같은 Vercel 프로젝트의 Python 함수(`api/ai/index.py` , FastAPI + LangGraph)이고 LLM은 엔노이아입니다. 설계는 `docs/ai_agent/에이전트 설계 확정.md` (D-302~D-326). 답하는 시간은 2026-10-27 확정한 **목표 값**이고, 우리 서버 코드는 개발 1단계에서 이 값으로 바꿉니다(D-309).
>
> 기준: 우리 코드 `shared/providers/types.ts` (2026-10-27, staging). 배경·흐름·검사 규칙의 자세한 설명은 `docs/API-SPEC.md` , 요청·응답 원본 예시는 `docs/api-examples/` 6개(코드가 만든 값)에 있습니다. 회사·공고·인물은 시연용 가명입니다.

API 목록

번호	이름	Method · URL	답하는 시간
1	1차 질문 문장 생성	`POST {AI_ENDPOINT}/v1/first-questions`	18초 안
2-1	최종 면접: 1차 면접 답 판정	`POST {AI_ENDPOINT}/v1/final-judgements`	8초 안
2-2	최종 면접·처우 질문 문장 생성	`POST {AI_ENDPOINT}/v1/final-questions`	10초 안
3	공고 분석 (카드 상태·처우 빈칸·회사 신호)	`POST {AI_ENDPOINT}/v1/posting-analysis`	함수 한도 안(공고당 한 번, 결과 저장)
4	업무 카드 초안	`POST {AI_ENDPOINT}/v1/card-drafts`	지금은 CLI(`npm run cards:draft`)가 지원 경로. 엔드포인트는 시간 한계(55초)에 걸려 503일 수 있음(4.5단계에서 총 168.2초로 재어 CLI 전용으로 정함, D-368)
내부	모델 미리 깨우기(화면을 열 때 우리 서버가 부름, 요청 본문 없음)	`POST {AI_ENDPOINT}/v1/warmup`	15초 안, 200 `{"ok": true 또는 false}`

최종 면접은 2-1 → (우리 규칙이 질문 순서 결정) → 2-2 순서로 두 번 부릅니다. 화면이 지금 쓰는 것은 1·2번이고, 3·4번은 지금 개발자가 부르며 나중에 화면이 필요하면 우리 서버가 대신 부릅니다.

공통

항목	값
Method	`POST`

항목	값
URL (운영)	`{AI_ENDPOINT}/v1/<호출>` — `AI_ENDPOINT` 는 같은 배포의 `https://<배포 주소>/api/ai` (비어 있으면 `https://${VERCEL_URL}/api/ai`). **https만** 부릅니다. 미리보기 배포에서는 `x-vercel-protection-bypass` 머리글을 함께 보냅니다
URL (테스트)	로컬 시험은 `http://localhost:<port>` · `http://127.0.0.1:<port>` 도 됩니다
`Authorization`	`Bearer {AI_API_KEY}` — 키는 별도 전달(우리 서버 환경변수에만 둠). 운영에서 키가 없으면 우리 서버가 부르지 않습니다
`Content-Type`	`application/json; charset=utf-8` (응답도 UTF-8 JSON 하나, 한글 그대로)
시간 제한	우리 서버는 요청 하나에 **20초**에 끊고 다시 부르지 않습니다. 최종은 2-1과 2-2가 20초를 나눠 씁니다(8초 + 10초 권장). AI 서버는 검사에 걸린 문항만, 자기 몫의 시간에 여유가 있으면 한 번 더 써도 됩니다
개인정보	요청·응답 본문을 저장·기록·학습에 쓰지 않습니다. 로그에는 `card_id` / `key` , 글자 수, 처리 시간, 모델 이름만 남깁니다. 경험 한 줄(`experience`)과 들은 답(`heard`)은 특히 저장하지 않습니다

오류

처리할 수 없으면 HTTP 4xx·5xx와 아래 형식으로 답합니다. `message` 에 요청 내용을 넣지 않습니다.

HTTP	code	message (예)
400	`invalid_request`	items is empty (본문 모양이 틀림)
401	`unauthorized`	invalid api key (키가 없거나 틀림)
429	`rate_limited`	too many requests
500	`internal`	internal error
503	`unavailable`	model is not ready (모델 준비 중·과부하)

실패 예시 (401)

```
{
  "error": {
    "code": "unauthorized",
    "message": "invalid api key"
  }
}
```

우리 서버는 오류 본문을 읽지 않고 상태만 봅니다. 오류·시간 초과·JSON 아님이면 그 호출의 문항을 모두 미리 검수한 문장(검수 문장)으로 바꾸고, 문항 하나만 틀리면 그 문항만 바꿉니다. 화면은 멈추지 않습니다.

1. 1차 질문 문장 생성 API

1차 면접에서 물을 질문 5개의 문장을 씁니다. 순서·유형·근거는 요청에 담긴 대로 두고, 검수 문장(`template_text`)의 뜻을 바꾸지 않고 면접에서 그대로 말할 문장으로 다듬습니다.

요청

항목	값
Method	`POST`
URL	`{AI_ENDPOINT}/v1/first-questions`
`Authorization`	`Bearer {AI_API_KEY}`

Body	필수	타입	설명
`mode`	Y	string	`"first"`
`posting`	Y	object	지원한 공고 원문 한 건 전체(「공고 posting 필드」)
`experience`	Y	string	경험 한 줄(100자 이내). 화면 ② "경험 한 줄" 입력 또는 이력서 경험 칩. 붙일 질문이 없으면 `""`
`items`	Y	array	질문 5개(우리 규칙이 고른 상위 5개, 순위 순)
`items[].rank`	Y	number	순위 1~5
`items[].card_id`	Y	string	업무 카드 ID(`BE01` ~ `BE17`). 응답에 그대로 돌려줌
`items[].card_label`	Y	string	카드 이름(예: 고객사 상주 개발)
`items[].pref`	Y	string	화면 ② 카드에서 고른 것: `want` · `avoid` · `unsure` (「값 목록」)
`items[].state`	Y	string	공고가 이 업무를 다루는 상태: `explicit` · `inferred` · `vague` · `absent`
`items[].type`	Y	string	질문 유형: `conflict` · `opportunity` · `depth` · `prevent` · `explore`
`items[].evidence`	N	object	공고 근거 인용 `{ "field", "quote" }` . 없으면 키가 없음
`items[].evidence.field`	Y	string	근거가 있는 공고 칸(8개, 「공고 posting 필드」)
`items[].evidence.quote`	Y	string	`posting[field]` 원문의 글자 그대로 일부
`items[].template`	Y	string	질문 틀. `{anchor}` 자리에 경험 한 줄이 들어감
`items[].template_text`	Y	string	검수 문장(틀 + 경험). AI가 실패하면 이 문장이 나가며, 뜻의 기준
`items[].has_exp`	Y	boolean	`true` 면 이 질문에만 경험 한 줄을 살려 씀

```
curl -X POST "{AI_ENDPOINT}/v1/first-questions" \
  -H "Authorization: Bearer {AI_API_KEY}" \
  -H "Content-Type: application/json; charset=utf-8" \
  -d @docs/api-examples/first-request.json
```

요청 예시 (`posting` 과 `items` 는 일부만 — 전체는 `docs/api-examples/first-request.json`)

```
{
  "mode": "first",
  "posting": {
    "wantedAuthNo": "K-MOCK-0001",
    "corpNm": "(주)OO소프트",
    "wantedTitle": "백엔드 개발자 (신입·경력 1년 이하)",
    "jobCont": "[회사소개]\n(주)OO소프트는 공공기관과 금융권의 정보시스템 구축 사업을 수행하는 IT 서비스 기업입니다. …",
    "pfCond": "고객사 상주 가능자, 정보처리기사",
    "selMthd": "서류전형, 1차 면접(실무), 최종 면접(임원)",
    "…": "…"
  },
  "experience": "직업훈련 팀 프로젝트에서 FastAPI로 API를 설계부터 배포까지 맡아 본 경험이 있습니다.",
  "items": [
    {
      "rank": 1,
      "card_id": "BE15",
      "card_label": "고객사 상주 개발",
      "pref": "avoid",
      "state": "explicit",
      "type": "conflict",
      "template": "신입이 고객사에 상주해 일하는 비율과, 한 번 나가면 보통 기간은 어느 정도인가요?",
      "template_text": "신입이 고객사에 상주해 일하는 비율과, 한 번 나가면 보통 기간은 어느 정도인가요?",
      "has_exp": false,
      "evidence": { "field": "pfCond", "quote": "고객사 상주 가능자" }
    }
  ]
}
```

응답

```
{
  "questions": [
    {
      "card_id": "BE15",
      "text": "공고의 '고객사 상주 가능자' 부분과 관련해 확인하고 싶은 점이 있습니다. 신입이 고객사에 상주해 일하는 비율과, 한 번 나가면 보통 기간은 어느 정도인가요?",
      "evidence": { "field": "pfCond", "quote": "고객사 상주 가능자" }
    },
    {
      "card_id": "BE07",
      "text": "공고에 적힌 '기타 업무 지원'에 어떤 업무까지 포함되는지 여쭙고 싶습니다. 현재 팀 업무에서 기존 시스템 유지보수와 새 기능 개발의 비중은 대략 어떻게 되나요?",
      "evidence": { "field": "jobCont", "quote": "기타 업무 지원" }
    },
    {
      "card_id": "BE02",
      "text": "제가 특히 관심을 두고 있는 업무입니다. 설계 회의나 설계 리뷰에 신입도 참여하나요? 참여한다면 어떤 역할을 맡게 되나요?"
    },
    {
      "card_id": "BE03",
      "text": "입사하면 꼭 해 보고 싶은 업무라 여쭙겠습니다. DB 테이블 설계는 백엔드 개발자가 직접 하나요, 별도 담당자가 있나요?"
    },
    {
      "card_id": "BE01",
      "text": "직업훈련 팀 프로젝트에서 FastAPI로 API를 설계부터 배포까지 맡아 본 경험이 있습니다. 그때 배운 것을 살릴 수 있을지 여쭙고 싶습니다. 이 팀에서는 새 API를 만들 때 신입이 어디까지 맡나요? 설계부터 배포까지인지, 정해진 스펙의 구현까지인지 궁금합니다.",
```

```
      "evidence": { "field": "jobCont", "quote": "서버 개발" }
    }
  ],
  "model": "mock"
}
```

필드	타입	설명
`questions`	array	요청 `items` 순서대로, 검사를 통과한 문항만(빠진 문항은 우리 서버가 검수 문장으로 채움)
`questions[].card_id`	string	요청의 `card_id` 그대로. 필수
`questions[].text`	string	질문 문장(10~260자, 면접 비즈니스체). 필수
`questions[].evidence`	object	선택. 돌려준다면 요청 값 그대로(`null` 도 됨). 우리 서버는 요청의 근거를 씀
`questions[].evidence.field`	string	요청 값 그대로
`questions[].evidence.quote`	string	요청 값 그대로
`model`	string	선택. 사용한 모델 이름(문제를 찾을 때 봄, 우리 서버는 읽지 않음)

값 목록

pref	**화면 ② 카드에서 고른 것**
`want`	기대돼요
`avoid`	걱정돼요
`unsure`	모르겠어요

state	**공고가 이 업무를 다루는 상태**
`explicit`	공고에 적혀 있음
`inferred`	공고에서 유추
`vague`	모호한 표현
`absent`	공고에 없음 — "공고에 … 있던데"처럼 전제하면 안 됨

type	**질문 유형**	**언제**
`conflict`	차이 확인	걱정한 일이 공고에 있거나 포함될 수 있어 범위를 확인
`opportunity`	기회 확인	기대한 일이 공고에 분명하지 않아 맡을 기회를 확인
`depth`	깊이 확인	기대한 일이 공고에 있어 얼마나 깊이 맡는지 확인
`prevent`	미리 확인	걱정한 일이 공고에 없어 생길 수 있는지 확인
`explore`	탐색	잘 모르는 일이라 어떤 일인지 알아봄

2. 최종 면접·처우 질문 생성 API

1차 면접을 다녀와 정리한 내용으로 최종 면접·처우 협의 질문을 만듭니다. **2-1 판정**을 받은 뒤 우리 규칙이 질문 순서·처우 질문·완충 문장을 정하고, 그 순서로 **2-2 문장**을 부탁합니다.

2-1. 1차 면접 답 판정

1차 면접에서 물어보고 들은 답이 내 기대·걱정과 맞는지 카드마다 판정합니다. 물어봤고 들은 답이 있는 카드만 보냅니다.

요청

항목	값
Method	`POST`
URL	`{AI_ENDPOINT}/v1/final-judgements`
`Authorization`	`Bearer {AI_API_KEY}`

Body	필수	타입	설명
`mode`	Y	string	`"final"`
`posting`	Y	object	지원한 공고 원문 한 건 전체
`items`	Y	array	판정할 카드(1차 면접 정리에서 "물어봤어요"이고 들은 답이 있는 것)
`items[].card_id`	Y	string	카드 ID. 응답에 그대로
`items[].card_label`	Y	string	카드 이름
`items[].pref`	Y	string	`want` · `avoid` · `unsure` · `none` (고르지 않은 카드)
`items[].heard`	Y	string	1차 면접 정리 "들은 답"(100자 이내). 판정에만 쓰고 저장하지 않음
`items[].clarity`	N	string	1차 면접 정리 "구체적이었어요" `clear` · "모호했어요" `vague`. 안 골랐으면 키가 없음
`items[].watch_out`	Y	string	이 카드의 주의할 답(기대와 다르거나 걱정이 맞는 답). 판정 기준
`items[].good_sign`	Y	string	이 카드의 좋은 신호. 판정 기준

```
curl -X POST "{AI_ENDPOINT}/v1/final-judgements" \
  -H "Authorization: Bearer {AI_API_KEY}" \
  -H "Content-Type: application/json; charset=utf-8" \
  -d @docs/api-examples/final-judgements-request.json
```

요청 예시 (`posting` 과 `items` 는 일부만 — 전체는 `docs/api-examples/final-judgements-request.json`)

```
{
  "mode": "final",
  "posting": { "wantedAuthNo": "K-MOCK-0001", "corpNm": "(주)OO소프트", "…": "…" },
  "items": [
    {
      "card_id": "BE15",
      "card_label": "고객사 상주 개발",
      "pref": "avoid",
```

```
      "heard": "프로젝트마다 달라서요",
      "watch_out": "상주 비중이 절반 이상이거나 '프로젝트마다 달라요'만 되풀이해요",
      "good_sign": "비중·기간·배정 방식을 구체적으로 설명해요",
      "clarity": "vague"
    },
    {
      "card_id": "BE07",
      "card_label": "기존 시스템 유지보수",
      "pref": "avoid",
      "heard": "유지보수가 7할쯤 되고 신규는 프로젝트 들어갈 때만요",
      "watch_out": "유지보수가 7할 이상이거나 비중을 구체적으로 말하지 못해요",
      "good_sign": "유지보수와 신규 개발의 비중을 구체적으로 말하고 돌아가며 맡을 기회가 있어요",
      "clarity": "clear"
    }
  ]
}
```

응답

```
{
  "judgements": [
    {
      "card_id": "BE15",
      "state": "still_unknown",
      "basis": "답이 모호해 아직 확인되지 않았어요"
    },
    {
      "card_id": "BE07",
      "state": "mismatch",
      "basis": "들은 답이 이 카드의 주의할 답과 맞아 걱정한 대로예요 · 주의할 답: 유지보수가 7할 이상이거나 비중을 구체적으로 말하지 못해요"
    },
    {
      "card_id": "BE02",
      "state": "confirmed",
      "basis": "들은 답이 이 카드의 좋은 신호와 맞아 확인됐어요 · 좋은 신호: 설계 리뷰 참여나 작은 기능 설계를 맡긴다고 구체적으로 답해요"
    },
    {
      "card_id": "BE01",
      "state": "confirmed",
      "basis": "들은 답이 이 카드의 좋은 신호와 맞아 확인됐어요 · 좋은 신호: 수습이 끝나면 작은 기능부터 혼자 맡은 사례를 구체적으로 들려줘요"
    }
  ],
  "model": "mock"
}
```

필드	타입	설명
`judgements`	array	요청 `items` 순서대로, 검사를 통과한 판정만. 판단이 어려운 카드는 빼도 됨(빠진 카드는 우리 서버가 단순 규칙으로 판정)
`judgements[].card_id`	string	요청의 `card_id` 그대로. 필수
`judgements[].state`	string	`confirmed` · `mismatch` · `still_unknown` (「판정 상태」). 필수
`judgements[].basis`	string	판정 근거 한 줄(1~160자, 해요체, 화면에 보임). 들은 답을 그대로 옮기지 않음. 필수
`model`	string	선택. 사용한 모델 이름

판정 상태

state	조건
`confirmed`	들은 답이 좋은 신호와 맞아 확인됨
`mismatch`	기대(`want`)와 다른 답이거나 걱정(`avoid`)이 맞는 답. 걱정한 카드면 `basis`도 "…걱정한 대로예요"처럼 씀(화면은 "걱정한 대로"로 보임)
`still_unknown`	답이 모호하거나 판단할 수 없어 아직 모름

clarity	화면 1차 면접 정리
`clear`	구체적이었어요
`vague`	모호했어요

2-2. 최종 면접·처우 질문 문장 생성

2-1 판정과 우리 순서 규칙으로 고른 질문(카드 질문 5개 이하 + 처우 질문 3개 이하, 합쳐 8개 이하)의 문장을 씁니다. 1차 면접관의 말을 옮기지 않도록 **들은 답은 보내지 않습니다.**

요청

항목	값
Method	`POST`
URL	`{AI_ENDPOINT}/v1/final-questions`
`Authorization`	`Bearer {AI_API_KEY}`

Body	필수	타입	설명
`mode`	Y	string	`"final"`
`posting`	Y	object	지원한 공고 원문 한 건 전체
`items`	Y	array	질문 8개 이하(우리 규칙 순서)
`items[].rank`	Y	number	순위 1~8
`items[].key`	Y	string	카드 질문은 카드 ID, 처우 질문은 `offer:<topic>`. 응답에 그대로
`items[].card_id`	N	string	카드 질문만
`items[].topic`	N	string	처우 질문만: `sal` · `workHours` · `probation` · `onsite`
`items[].label`	Y	string	카드 이름 또는 근로조건 이름(예: 근로시간)
`items[].type`	Y	string	`recheck` · `signal` · `scope` · `offer` · `opportunity`
`items[].origin`	Y	string	이 질문이 나온 곳: `asked` · `not_asked` · `received` · `offer`
`items[].state`	N	string	카드 질문만: 공고 상태(1번 API와 같은 값)

Body	필수	타입	설명
`items[].review_state`	N	string	카드 질문만: 1차 면접 뒤 상태 `confirmed` · `mismatch` · `still_unknown` · `company_signal`
`items[].evidence`	N	object	공고 근거 인용 `{ "field", "quote" }` (처우 질문 포함, 있을 때만)
`items[].tone`	Y	string	표현 단계 `basic` · `cushion` · `direct`
`items[].cushion`	Y	string	완충 문장(`tone` 이 `cushion` 일 때만, 아니면 `""`). 문장 맨 앞에 그대로
`items[].template`	Y	string	완충을 뺀 검수 문장
`items[].template_text`	Y	string	완충 + 검수 문장. AI가 실패하면 이 문장이 나감

```
curl -X POST "{AI_ENDPOINT}/v1/final-questions" \
  -H "Authorization: Bearer {AI_API_KEY}" \
  -H "Content-Type: application/json; charset=utf-8" \
  -d @docs/api-examples/final-questions-request.json
```

요청 예시 (`posting` 과 `items` 는 일부만 — 전체는 `docs/api-examples/final-questions-request.json`)

```
{
  "mode": "final",
  "posting": { "wantedAuthNo": "K-MOCK-0001", "corpNm": "(주)OO소프트", "…": "…" },
  "items": [
    {
      "rank": 1,
      "key": "BE15",
      "label": "고객사 상주 개발",
      "type": "recheck",
      "origin": "asked",
      "tone": "cushion",
      "cushion": "괜찮으시다면 여쭤도 될까요? ",
      "template": "지난 면접에 이어 공고의 '고객사 상주 가능자' 부분을 조금 더 구체적으로 여쭙겠습니다. 첫해에 신입이 상주 프로젝트에 나간 기간이 보통 어느 정도였는지 예를 들어 주실 수 있을까요?",
      "template_text": "괜찮으시다면 여쭤도 될까요? 지난 면접에 이어 공고의 '고객사 상주 가능자' 부분을 조금 더 구체적으로 여쭙겠습니다. 첫해에 신입이 상주 프로젝트에 나간 기간이 보통 어느 정도였는지 예를 들어 주실 수 있을까요?",
      "card_id": "BE15",
      "state": "explicit",
      "review_state": "still_unknown",
      "evidence": { "field": "pfCond", "quote": "고객사 상주 가능자" }
    },
    {
      "rank": 4,
      "key": "offer:workHours",
      "label": "근로시간",
      "type": "offer",
      "origin": "offer",
      "tone": "cushion",
      "cushion": "조심스럽지만 이어서 여쭤도 될까요? ",
      "template": "근무 시간이 공고에 적혀 있지 않은데, 출퇴근 시간과 주 근로시간은 어떻게 되고 연장근로가 있다면 월 평균 어느 정도인가요?",
      "template_text": "조심스럽지만 이어서 여쭤도 될까요? 근무 시간이 공고에 적혀 있지 않은데, 출퇴근 시간과 주 근로시간은 어떻게 되고 연장근로가 있다면 월 평균 어느 정도인가요?",
      "topic": "workHours"
    }
  ]
}
```

응답


```
{
  "questions": [
    {
      "key": "BE15",
      "text": "괜찮으시다면 여쭤도 될까요? 지난 면접에 이어 공고의 '고객사 상주 가능자' 부분을 조금 더 구체적으로 여쭙겠습니다. 첫해에 신입이 상주 프로젝트에 나간 기간이 보통 어느 정도였는지 예를 들어 주실 수 있을까요?",
      "evidence": { "field": "pfCond", "quote": "고객사 상주 가능자" }
    },
    {
      "key": "BE08",
      "text": "입사 전에 미리 준비해 두고 싶은 부분이 있습니다. 혹시 함께하게 된다면 장애 대응·온콜 업무도 맡게 될 수 있는지 여쭙고 싶습니다. 장애 대응 당번(온콜)이 있다면 신입도 들어가나요? 들어간다면 주기는 어느 정도인가요?"
    },
    {
      "key": "offer:workHours",
      "text": "조심스럽지만 이어서 여쭤도 될까요? 근무 시간이 공고에 적혀 있지 않은데, 출퇴근 시간과 주 근로시간은 어떻게 되고 연장근로가 있다면 월 평균 어느 정도인가요?"
    },
    {
      "key": "offer:sal",
      "text": "한 가지 더 확인하고 싶은 조건이 있습니다. 제시된 연봉은 기본급 기준인가요, 연장근로 수당 등이 포함된 금액인가요?",
      "evidence": { "field": "sal", "quote": "3,200만 원 ~ 3,800만 원" }
    }
  ],
  "model": "mock"
}
```


(응답 예시는 7개 중 4개 — 전체는 `docs/api-examples/final-questions-response.json`)

필드	타입	설명
`questions`	array	요청 `items` 순서대로, 검사를 통과한 문항만(빠진 문항은 우리 서버가 검수 문장으로 채움)
`questions[].key`	string	요청의 `key` 그대로. 필수
`questions[].text`	string	질문 문장(10~260자, 면접 비즈니스체). `cushion`이 있으면 맨 앞에 그대로. 필수
`questions[].evidence`	object	선택. 돌려준다면 요청 값 그대로. 우리 서버는 요청의 근거를 씀
`questions[].evidence.field`	string	요청 값 그대로
`questions[].evidence.quote`	string	요청 값 그대로
`model`	string	선택. 사용한 모델 이름

값 목록

type	질문 유형	언제
`recheck`	다시 확인	1차 면접에서 아직 확인되지 않은 것을 이어서 다시 물음
`signal`	신호 확인	회사가 먼저 물은 업무라 실제로 맡게 될지 확인
`scope`	범위 확인	들은 답이 기대와 다르거나 걱정한 대로라 맡을 범위를 확인
`offer`	처우 확인	근무시간·임금 같은 근무 조건을 확인
`opportunity`	기회 확인	회사가 먼저 물은 업무가 기대한 일이라 맡을 기회를 확인

origin	이 질문이 나온 곳(화면 1차 면접 정리)
`asked`	물어봤어요
`not_asked`	못 물어봤어요
`received`	회사가 물어본 업무(카드 체크)
`offer`	처우 질문(공고에서 빠졌거나 모호한 근로조건)

tone	표현 단계(화면 "면접 분위기"·못 물은 이유로 정함)
`basic`	기본 — 앞말을 붙여도 됨
`cushion`	완충 — `cushion` 문장을 맨 앞에 그대로 두고, 앞말은 더하지 않음("압박이 있었어요"·"분위기가 아니었어요")
`direct`	직접 — 앞말 없이 바로 물음

topic	처우 질문 근로조건
`sal`	임금 구성
`workHours`	근로시간
`probation`	수습
`onsite`	상주 근무 조건

3. 공고 분석 API

공고 한 건을 업무 카드 사전과 비교해 카드마다 공고 상태·근거·이유를 붙이고, 처우 빈칸과 회사 신호(신호 질문 포함)를 만듭니다. 공고당 한 번 돌려 결과를 저장합니다(지금은 `data/analysis/<공고>.json` · `data/signals.json`에 커밋). 질문 순서는 정하지 않습니다 — 순서는 우리 규칙이 상태로 계산합니다.

요청

항목	값
Method	`POST`
URL	`{AI_ENDPOINT}/v1/posting-analysis`
`Authorization`	`Bearer {AI_API_KEY}`

Body	필수	타입	설명
`posting`	Y	object	공고 원문 한 건 전체(「공고 posting 필드」). 실서비스의 실제 공고면 `bizNo`(사업자등록번호)를 더함(선택)
`job`	Y	string	직무 ID(예: `backend`). 이 직무의 업무 카드 사전(`data/cards.<job>.json`)과 비교

회사 자료는 요청에 담지 않습니다. AI 서버가 회사를 이 순서로 찾습니다: 서버 환경변수 `REAL_COMPANY_MAP`(시연 공고 번호 → 실제 회사, 팀 확인 전에는 비어 있음) → 공고의 `bizNo` → 시연 모의 값. 국민연금(data.go.kr 오픈API)은 실시간으로 부르고, 강소기업·EIS 업종 기준값·임금체불(모의)·공고 등록 이력은 함께 올린 `data/company/` 파일에서 읽습니다. EIS 칸이 비었을 때 EIS Open API로 실시간 채우는 일은 응답 형식을 확인한 뒤로 미뤘고(D-355), 지금은 파일 값만 씁니다. 공고에 `bizNo`만 있고 사업장명이 없으면 국민연금 검색을 할 수 없어 그 신호는 뺍니다(실서비스 공고는 사업장명 `corpNm`을 검색 이름으로 쓰는 일이 남아 있습니다). 외부 API가 실패하면 그 신호만 뺍니다. **실제 이름·사업자번호는 응답에 넣지 않습니다.**

응답

필드	타입	설명
`analysis.wantedAuthNo`	string	요청 공고 번호
`analysis.cards`	array	사전의 카드 전부, 하나씩(사전 순서). `absent`도 포함. 저장 파일(`data/analysis/`)에는 `absent` 카드를 쓰지 않는다
`analysis.cards[].card_id`	string	사전에 있는 카드 ID
`analysis.cards[].state`	string	`explicit` · `inferred` · `vague` · `absent`
`analysis.cards[].evidence`	object	`explicit`·`vague`는 필수, `inferred`는 선택, `absent`는 없음. 있으면 `{ "field", "quote" }` — `quote`는 `posting[field]` 원문의 글자 그대로 일부(근거 칸 8개 중)
`analysis.cards[].reason`	string	화면 말풍선. 과정이 아니라 확인해 볼 이유(해요체)
`analysis.cards[].needs_review`	boolean	독립 2회 판정이 엇갈려 사람이 볼 카드
`analysis.blanks`	array	처우 빈칸(코드로 만듦)
`analysis.blanks[].topic`	string	`sal` · `workHours` · `probation` · `onsite`
`analysis.blanks[].status`	string	`missing`(공고에 없음) · `vague`(모호한 표현) · `check`(적혀 있지만 구성 확인 필요)
`analysis.blanks[].label` · `basis` · `question`	string	근로조건 이름 · 그렇게 본 까닭 · 검수 처우 질문
`analysis.blanks[].evidence`	object	있을 때만 `{ "field", "quote" }`
`signals`	array	켜진 회사 신호(켜는 규칙은 설계 확정 문서 2절 표)

필드	타입	설명
`signals[].kind`	string	`reposting` · `pension_flow` · `industry_baseline` · `small_giant` · `wage_arrears`
`signals[].text`	string	코드가 만든 사실 문장. 숫자는 여기에만, 실제 회사 값은 구간·방향만(예: 가입자 30~99명)
`signals[].source` · `source_name`	string	`실제` · `모의`, 데이터 이름
`signals[].ask`	string	신호 질문. 숫자·회사 평가 없음. 강소기업은 없음
`signals[].ask_source`	string	`ai` (검사 통과) · `template` (신호 종류별 검수 질문)
`signals[].caveat`	string	기준 시점·해석 한계(있을 때만). 매핑된 실제 값이면 "실제 공공데이터 값을 시연용 가상 회사에 붙여 보여 줘요"
`out_of_dictionary`	array	사전에 없는 업무가 공고에 나온 곳 `{ "field", "quote", "note" }` (사전 보강 참고, 질문에 안 씀)
`meta.graph_version` · `meta.model` · `meta.created_at`	string	만든 그래프 버전 · 모델 · 시각

4. 업무 카드 초안 API

직무 하나의 업무 카드 사전 **초안**을 만듭니다. NCS 능력단위 연결은 규칙이, 카드 글은 LLM이 쓰고, 사람이 검수해 `data/cards.<job>.json`으로 커밋합니다. 응답을 저장하지 않습니다.

요청

항목	값
Method	`POST`
URL	`{AI_ENDPOINT}/v1/card-drafts`
`Authorization`	`Bearer {AI_API_KEY}`

Body	필수	타입	설명
`job`	Y	string	새 직무 ID(영문 소문자만, `^[a-z]+$`. 예: `frontend`)
`job_label`	Y	string	직무 이름(예: 프론트엔드 개발자)
`ncs_subcategories`	Y	array	NCS 세분류 코드 목록(1~5개, 각 8자리 숫자. 예: 응용SW엔지니어링 `20010202`)
`job_dictionary_code`	N	string	한국직업사전 세세분류 코드(수행직무 문구를 붙일 때)
`max_cards`	N	number	묶은 뒤 카드 수 상한(1~30, 기본 15)

응답

필드	타입	설명
`candidates`	array	규칙이 만든 카드 후보(능력단위 하나에 하나)

필드	타입	설명
`candidates[].ablt_unit` · `unit` · `subcategory`	string	능력단위 코드 · 이름 · 세분류 이름
`candidates[].level`	number	능력단위 수준(알 수 있을 때). 신입과 먼 수준은 `high_level: true` 표시만
`candidates[].used_in`	string	묶인 카드 ID, 안 쓰였으면 `null`
`cards`	array	카드 초안(`shared/contract.ts` `CardDef` 모양: `id` · `label` · `plain` · `examples` · `ncs` · `posting_signals` · `questions` · `interpretation` …). 근무 방식 카드는 사람이 더함
`cards[].ncs[].ablt_unit`	string	이 카드에 묶인 능력단위 코드(묶음 근거)
`cards[].ncs[].subcategory` · `unit`	string	묶인 능력단위의 세분류 이름 · 이름
`cards[].job_dictionary` · `cards[].example_question`	string	없으면 `null` 이 아니라 필드를 뺀다. 직업사전 문구는 카드별 수행직무가 아니라 직업 요약이라 `meta.job_dictionary` 에 둔다
`issues`	array	검사에 걸린 곳 `{ "card_id", "rule" }` (금지어, 질문 틀에 "공고" 등)
`meta.graph_version` · `meta.model` · `meta.created_at`	string	만든 그래프 버전 · 모델 · 시각
`meta.job_dictionary`	string	직업사전(212D01)의 직업 한 줄 요약. `job_dictionary_code` 가 없거나 못 찾으면 `null`
`meta.sources`	object	실제 부른 자료 출처별 `실제` · `실제 스냅숏` · `모의` (키는 `ncs_gw` · `duty` · `job` 중 부른 것만, 예: `ncs_gw: "모의"`). 모의를 실제로 알고 검수하지 않게 한다

카드 ID는 서버가 `job` 앞 두 글자 대문자 + 두 자리 번호(`FR01`)로 정하고, 앞 12개는 `core: true` , `mapping_confidence` 는 늘 `draft` 입니다. 능력단위를 하나도 찾지 못하면 400 `invalid_request` (`no units`)입니다. 키가 있는 자료의 호출이 실패하거나 마감 안에 답하지 않으면 503 `unavailable` (`source is not available`), 모델이 못 쓰거나 빈 초안이면 503(`model is not ready`)입니다.

시간: 엔드포인트는 전체 55초, LLM 한 번 45초 안에서 끝나야 합니다. 정확 모델의 실제 묶기 호출이 보통 그보다 길어(실측: 엔드포인트 45.7초에 timeout, CLI는 한도 170초로 66.6초에 성공) 지금은 엔드포인트가 503으로 끝날 수 있습니다. 직무를 넓힐 때는 CLI `npm run cards:draft` 를 쓰고, 4.5단계에서 실제 출처로 잰 총 시간이 168.2초(엔드포인트 55초의 약 3배)라, 엔드포인트는 남기되 지금 모델로는 지원 경로가 아니고 ⓪은 CLI 전용으로 정했습니다(D-368).

유형 이름·규칙표(`type_labels` · `gap_rules`)는 직무와 상관없어 초안에 넣지 않고 기존 사전의 것을 씁니다.

공고 posting 필드

1·2·3번 API 모두 공고 한 건을 그대로 보냅니다. 필드 이름은 고용24 채용정보 API 상세 명세를 따릅니다. 값은 모두 문자열이고(`draft` 만 boolean), **빈 문자열은 공고에 적혀 있지 않다는 뜻**입니다.

필드	타입	설명
`wantedAuthNo`	string	공고 번호(시연 `K-MOCK-0001` ~ `0007`)
`draft`	boolean	우리 쪽 표시: 임시 공고 여부
`corpNm`	string	회사명
`indTpNm`	string	업종
`totPsncnt`	string	근로자 수
`busiSize`	string	기업 규모
`busiCont`	string	주요 사업 내용
`wantedTitle`	string	채용 제목
`jobsNm`	string	모집 직종
`jobsCd`	string	직종 코드(고용24 공통코드)
`jobCont`	string	직무 내용(긴 글, 줄바꿈 포함) · 근거 칸
`empTpNm`	string	고용 형태 · 근거 칸
`collectPsncnt`	string	모집 인원
`salTpNm`	string	임금 형태 · 근거 칸
`sal`	string	임금 · 근거 칸
`enterTpNm`	string	경력 조건
`eduNm`	string	학력
`pfCond`	string	우대 조건 · 근거 칸
`etcPfCond`	string	기타 우대 조건 · 근거 칸
`selMthd`	string	전형 방법
`workRegion`	string	근무 지역
`regionCd`	string	지역 코드(고용24 공통코드)
`workdayWorkhrCont`	string	근무 시간·형태 · 근거 칸
`fourIns`	string	4대 보험
`retirepay`	string	퇴직금
`etcWelfare`	string	복리후생 · 근거 칸
`receiptCloseDt`	string	접수 마감

근거 칸(`evidence.field`)은 `jobCont` · `pfCond` · `etcPfCond` · `empTpNm` · `salTpNm` · `sal` · `workdayWorkhrCont` · `etcWelfare` 8개입니다.

응답 규칙

규칙	우리 서버 검사	어기면
본문은 JSON 하나(설명 글·코드 울타리 없음)	검사함	그 호출 전체 검수 문장
요청 `items` 마다 하나씩, `card_id` · `key` 그대로, 요청 순서대로 통과한 것만	짝은 검사함	빠진 항목은 우리 서버가 검수 문장·단순 규칙으로 채움, 요청에 없는 항목은 버림
문장 속 인용은 작은따옴표('…')만, 공고 원문과 **글자 그대로**(카드 이름 인용은 예외)	검사함	그 문항 검수 문장
`state: "absent"` 항목에 "공고에 … 있던데/있는데"처럼 공고에 없는 업무를 전제하지 않음	이 꼴만 검사함	그 문항 검수 문장
금지어(나이·결혼·출신·종교·임신·외모·가족·정치 성향, 회사를 깎아내리는 말)	검사함	그 문항(판정은 그 카드) 검수 문장·단순 판정
최종: 1차 면접관 말을 옮기지 않음("아까" "하셨는데" "말씀하신", 6자 이상 들은 답 그대로)	검사함	그 문항 검수 문장
길이: 질문 10~260자, 판정 `basis` 1~160자	검사함	그 문항 검수 문장
문체: 서술 "…습니다", 질문 "…인가요?"(해요체 아님), `template_text` 뜻 그대로, `has_exp` 질문은 경험 문장을 살림, `cushion` · `tone` 지킴	검사 안 함	AI 담당이 지켜 주세요(그대로 화면에 나감)
판정 `basis` 에 들은 답을 옮기지 않고 이유만 해요체 한 줄로	통째로 넣은 것만 검사함	그 판정을 받지 않음